Senior Project 2

RESEARCH PROPOSAL

PROJECT TITLE
Mini-NPU: A CoralNPU-Inspired RISC-V Neural Processing Unit with a Decoupled int8 MAC Engine, Verified with a Self-Built UVM Environment and Demonstrated on an int8 MNIST Model

Field	Information
Student / Team	Nguyễn Hoàng Công Tấn / Nguyễn Thế Bảo
Student ID(s)	23161064 / 23144002
Program / Cohort	Electronics and Telecommunications Engineering Technology, class 23161FIE
Supervisor	Van-Khoa Pham (Dr.)
Research Area	Edge-AI accelerators (RISC-V NPU), RTL design and UVM verification
Project Duration	16 weeks (12 Oct 2026 – 31 Jan 2027)
Submission Date	11/11/2026

Required Research Logic

Stage	Core Question	Required Evidence
1. Technical Issues	What exactly is technically wrong, limited, costly, slow, inaccurate, unsafe, or unsolved?	Quantified problem, constraints, metrics, research questions
2. Related Works	How have existing studies or systems addressed the problem?	Critical comparison, results, limitations, research gap
3. Proposed Idea	What new or adapted idea is proposed to close the gap?	System concept, architecture, novelty, hypotheses
4. Methodology	How will the idea be implemented and objectively evaluated?	Implementation plan, baselines, experiments, metrics, analysis
5. Expected Results	What measurable outcomes and deliverables are expected?	Target performance, contributions, prototype/report/demo

Recommended Writing Standards

- Use precise engineering terminology and measurable performance indicators rather than general claims such as “better,” “faster,” or “more efficient.”
- Use peer-reviewed and technically credible sources. Prefer recent work for the state of the art while retaining seminal references when necessary.
- For every important claim, provide evidence through citation, measurement, benchmark data, or a clearly stated assumption.
- Define all acronyms at first use and use consistent symbols, units, dataset names, hardware specifications, and evaluation conditions.
- The proposed solution must directly address the research gap identified from the related works; the evaluation metrics must directly reflect the technical issues defined in Section 1.

Suggested Proposal Length

Component	Suggested Length
Core Sections 1-5	6-10 pages
Figures / Tables	As required for technical clarity
References	Normally 10+ credible sources for a research-oriented project
Appendices	Optional; implementation details, extended tables, risk register, etc.

1 TECHNICAL ISSUES / PROBLEM DESCRIPTION

Define the engineering problem in measurable and technically defensible terms.

1.1 Background and Technical Context

Machine-learning inference is moving from data centres to battery-powered, cost-constrained devices such as wearables, sensors, and smart-home endpoints. On these devices the workload is dominated by multiply-accumulate (MAC) operations on small integers, and post-training quantization to 8-bit integers (int8) preserves accuracy while allowing integer-only arithmetic [1]. Domain-specific accelerators exploit this regularity: the Tensor Processing Unit (TPU) uses a large 8-bit matrix unit [2], Eyeriss reduces data-movement energy with a

dedicated dataflow [3], and the survey in [4] shows that performance and energy are governed by data reuse and by how well the arithmetic units are kept busy. The observation that arithmetic costs far less energy than moving data [5] makes small, tightly-coupled memories and wide operand ports attractive for an edge neural processing unit (NPU).

Open-source projects make such architectures studyable. Google's Coral NPU combines a small RISC-V scalar core with vector and matrix execution units and tightly-coupled memories [6]; Gemmini generates systolic-array accelerators attached to RISC-V cores [7]. This project follows the Coral NPU architecture at a scale that a two-person team can design, verify, and explain completely. A 5-stage RV32IM core (with the Zicsr, Zifencei, and Zbb extensions [8]) is extended with four custom instructions, LOAD_WEIGHT, LOAD_INPUT, COMPUTE, and READ_RESULT, that push commands through an 8-entry FIFO to a decoupled int8 outer-product MAC engine of N×M×K = 4×4×4 (16 processing elements, each a 4-way dot product, 64 MACs per cycle). Core and engine share a 64 KB dual-port data memory (DTCM). The topic belongs to Computer Engineering because it joins instruction-set extension, microarchitecture, memory-system design, bare-metal firmware, and hardware verification in one system, the hardware/software co-design theme of domain-specific computing [9].

An NPU is only useful if the whole chain works together: instruction decode and command push, the command queue, the engine's state machine and error handling, the memory reads that feed the array, the accumulators, the read-back of results, and the firmware that orchestrates them. Quantized inference must also be exact at the integer level to be trustworthy, because a wrong command ordering, address-range check, accumulator wrap, or requantization step produces silently wrong predictions. Open and student designs are often shown with a few directed tests or waveforms; far fewer come with systematic verification evidence and an end-to-end model run that matches an independent golden model bit for bit. Industrial practice verifies such blocks with the Universal Verification Methodology (UVM), golden reference models, assertions, and functional coverage [10] [11], and the project builds that environment itself.

The demonstration workload is a multilayer perceptron (MLP) of size 784-64-32-10 on the MNIST handwritten-digit set [12], quantized to int8 with integer requantization between layers [1]. It is small enough to fit entirely in the 64 KB DTCM, yet it exercises every part of the NPU: weight and input loading, tiled matrix multiplication, accumulation, result read-back, and firmware requantization.

1.2 Technical Problem Statement

The technical problem addressed in this project is that there is no small, fully documented RISC-V NPU built on a CoralNPU-style architecture whose correctness is established by systematic verification and whose ability to run an AI model is demonstrated end to end against an independent golden model. Without such evidence, an NPU design cannot be trusted with real workloads. The project therefore has to (i) design the NPU (core extensions, custom instructions, command FIFO, MAC engine, dual-port memory), (ii) verify the engine, the core, and the whole SoC with a self-built UVM environment to measurable closure criteria, and (iii) run a quantized MLP on it with logits that match the golden model bit-exactly.

1.3 Technical Constraints and Operating Conditions

Constraint / Condition	Current or Required Value	Engineering Significance
Computing resources	QuestaSim 10.7c (UVM 1.2, DPI-C, coverage); Verilator 5.020 and RISC-V GNU toolchain under WSL2 (Ubuntu 24.04); Icarus Verilog 12 and GitHub Actions CI; Python 3 with numpy and PyTorch	QuestaSim runs the UVM flow. Its feature set (randomize, covergroup, DPI-C) was checked in week 1. Verilator is used for lint only.

Constraint / Condition	Current or Required Value	Engineering Significance
Memory / Storage	ITCM 16 KB; DTCM 64 KB (4 banks × 32 bit); WBUF and IBUF 256 × 128 bit each; command FIFO 8 entries; KMAX = 1024 elements; model data region 56,816 B	The whole MLP fits in the DTCM, so no DMA or external memory is needed.
Latency / Real-time requirement	No real-time deadline. Engine commands take K/4 + 2 busy cycles (bound K/4 + 3). Cycles per inference are reported as a measurement, with no preset target.	Shows how much of the run is spent in the engine and how much in scalar firmware.
Energy / Power	Not a project objective. No synthesis, technology library, or power estimation is used.	Power and energy work is deferred to the graduation thesis.
Accuracy / Quality	int8 MLP: 97.43 % on the 10,000 MNIST test images (float model 97.48 %); RTL logits bit-exact against the golden model on the first 1,000 images	Bit-exactness, not accuracy, is the success criterion; accuracy shows that quantization is not the limiting factor.
Communication / Bandwidth	DTCM port A 32 bit read/write (core); port B 128 bit read-only (engine), one quad of four words per cycle	Port B supplies one WBUF/IBUF entry per cycle, which keeps the 64-MAC PE array fed.
Cost / Area / Resource utilization	N×M×K = 4×4×4: 16 dot4 PEs (64 int8 multipliers); 8×8 only if time allows after the core criteria are met	Sets the engine size. Area is not evaluated because synthesis is outside the project.
Security / Privacy / Safety	Not a project objective. The engine drops illegal commands and records sticky error bits (err_k, err_addr, err_illegal, err_align).	Error handling is specified and verified, but no security analysis is made.

1.4 Research Questions

RQ1. Can a CoralNPU-inspired RISC-V NPU (RV32IM core, custom instructions, decoupled int8 MAC engine, dual-port DTCM) be designed so that it passes systematic verification with zero mismatches, zero assertion failures, and full functional coverage?

RQ2. How effectively can a self-built UVM environment with independent golden models, assertions, and coverage verify the engine, the core, and the whole SoC, and does it detect deliberately injected faults?

RQ3. Can the verified NPU run a quantized MLP (MNIST, 784-64-32-10) end to end with logits and labels that match the golden model bit-exactly, and what cycle count and engine busy fraction does one inference need?

Success Criteria

Criterion	Metric	Acceptance / Target Condition
SC1 Functional integrity	Scoreboard mismatches; assertion failures	Zero of both for every defined directed and random test of the engine, the core, and the SoC
SC2 Coverage	Functional coverage bins declared in the coverage plan	100 % of declared bins before the final measurements (week 10); every exclusion justified in writing
SC3 MNIST	RTL logits and labels versus the golden model	Bit-exact on the first 1,000 test images; the full 10,000-image run reproduces the golden accuracy
SC4 Reproducibility	Commands and recorded versions	One command per flow regenerates the regression, coverage, and MNIST results; tool versions and seeds are recorded

2 EXISTING SOLUTIONS AND RELATED WORKS

Establish the state of the art and derive a defensible research gap.

2.1 Literature Scope and Search Strategy

The review covers four themes that map to the project's contributions: (i) accelerator architectures that couple a matrix engine to a host core; (ii) open RISC-V NPUs and accelerator generators; (iii) integer quantization and the software that runs quantized models on small devices; and (iv) the verification methodology needed to prove an accelerator correct. Peer-reviewed work from 2014–2021 is used for accelerators and quantization, and the open-source Coral NPU repository (commit 382b5c1) and its developer guides, read on 15 October 2026, represent the current open RISC-V NPU. Standards (IEEE 1800, 1800.2, and the RISC-V ISA manual) and two textbooks [13][14] define the terminology.

2.2 Related Work Summary

Ref.	Problem / Context	Method / Architecture	Key Results	Limitations Relevant to This Project
[2] Jouppi et al. (2017)	Cost of neural-network inference in data centres	256×256 systolic matrix unit of 8-bit MACs with software-managed on-chip memory	Reported about 15–30× higher performance and 30–80× higher TOPS/W than a contemporary CPU and GPU on production workloads	Data-centre scale and fixed function; no RISC-V host and no open RTL or verification flow
[3] Chen et al. (2017)	Energy dominated by data movement in CNN accelerators	Row-stationary dataflow on 168 PEs in 65 nm CMOS	Silicon-measured energy efficiency from data reuse in the AlexNet convolution layers	Fixed-function CNN chip; no instruction-set interface to a host core; no open RTL or verification flow
[7] Genc et al. (2021)	Evaluating DNN accelerators together with their host system	Chisel generator of systolic-array accelerators attached to RISC-V cores, with a software stack	Full-system design-space exploration of accelerator and host	Aimed at design-space exploration rather than a small, fully verified reference design; large code base
[6] Google, Coral NPU	Open RISC-V NPU for edge AI	In-order scalar core, vector unit, and outer-product matrix unit; typically 8×8×4 MAC array; UVM and cocotb tests	Complete open system with developer guides and example ML workloads	Large Chisel code base; the matrix unit is marked experimental in the repository; hard to study end to end in one semester
[1] Jacob et al. (2018)	Running neural networks with integer-only arithmetic	int8 weights and activations, int32 accumulation, fixed-point requantization	Accuracy–latency gains for MobileNet inference on mobile CPUs	Software on CPUs; hardware execution and a bit-exact hardware golden model are left to the implementer
[15] David et al. (2021)	Running ML models on microcontrollers	Interpreter-based framework with portable kernels for embedded targets	Widely used framework for small devices	Software only; no custom instruction or dedicated engine; a compiler flow is outside this project

2.3 Critical Comparison of Existing Approaches

Approach	Main Idea	Advantages	Limitations	Evaluation Conditions / Performance
Method A Large dataflow and systolic accelerators ([2][3])	Maximise utilization and data reuse with a dedicated array and dataflow	High throughput per watt when the array is busy	Fixed function and large; no open verification flow; size exceeds a two-person project	ASIC, different nodes and workloads; numbers not directly comparable with each other or with this project

Approach	Main Idea	Advantages	Limitations	Evaluation Conditions / Performance
Method B Accelerator generators on RISC-V ([7])	Generate configurable systolic arrays attached to RISC-V cores	Flexible, with a software stack and full-system evaluation	Focused on design-space exploration; a small verified reference with bit-exact model execution is not its goal	Full-system benchmarks on FPGA or simulation; not directly comparable with an MNIST MLP
Method C Open RISC-V NPU ([6])	Scalar core plus decoupled matrix engine with tightly-coupled memories	Complete open system with a software stack; UVM structure to study	Very large code base; matrix unit marked experimental; not scaled for full study by a small team	Developer guides and design documents; not directly comparable with this project's cycle counts
Method D Software quantized inference ([1][15])	int8 data path with int32 accumulators and fixed-point rescaling, run by a software runtime	Preserves accuracy; runs on existing microcontrollers	No dedicated engine; hardware execution and its verification are out of scope	ImageNet MobileNet on mobile CPUs, microcontroller benchmarks; not directly comparable with an MNIST MLP on a custom NPU

2.4 Identified Research Gap

Existing designs are either large fixed-function accelerators [2][3], generators aimed at design-space exploration [7], or a large open NPU whose matrix engine is still experimental [6]; software stacks [1][15] run quantized models on microcontrollers without a dedicated engine. Within the sources reviewed here, none provides a small, fully documented NPU that combines three elements this project needs together: (a) a custom-instruction interface from an in-house core to a decoupled int8 outer-product engine; (b) verification by a UVM environment with independent golden models, assertions, and closed functional coverage; and (c) an end-to-end run of a quantized model whose results match the golden model bit-exactly.

This is a gap in verified, reproducible reference designs, not a claim of a new architecture. The architecture follows Coral NPU [6]; the contribution is a design that a small team can verify completely and use to run a real model.

2.5 Positioning Statement

Compared with the reviewed approaches, this project does not propose a larger accelerator [2][7] or a new dataflow [3]. Its contribution is a complete, reproducible, small-scale study: an NPU designed from the instruction set down to RTL on the semantics of Coral NPU [6] (outer-product N×M×K engine, dot4 processing elements, command queue, tightly-coupled memories), verified by a self-built UVM environment, and shown to run a quantized MNIST MLP bit-exactly. It differs from [6] by using its own 5-stage core, a single 4×4×4 tile, independent golden models in C and Python, and verification evidence that is regenerated by one command per flow, so that every design decision can be explained.

3 PROPOSED IDEA / PROPOSED SOLUTION

Describe the solution concept, architecture, novelty, and testable hypotheses.

3.1 Core Technical Idea

This project introduces a CoralNPU-inspired RISC-V NPU in which four custom-0 instructions push commands through an 8-entry FIFO to a decoupled int8 outer-product MAC engine (4×4×4 = 64 MACs per cycle) that reads its operands from a dual-port DTCM and accumulates in int32. The NPU is verified by three self-built UVM environments (engine, core, and SoC) that check every result against independent golden models, with assertions and functional coverage. Its capability is then demonstrated by running a quantized 784-64-32-10 MLP on MNIST with bare-metal firmware, whose logits must match the golden model bit-exactly. This addresses the gap of Section 2.4: a small NPU on a Coral-style architecture with systematic verification evidence and an end-to-end model run.

3.2 Proposed System Architecture

Architecture Element	Description / Interface / Function
Input / Data acquisition	Bare-metal firmware (C and assembly, custom instructions through `.insn` wrappers) and the int8 MLP data loaded into ITCM and DTCM by backdoor `$readmemh` before reset. MNIST pixels p are quantized as (p·254 + 255) // 510 to 0..127; the 784-64-32-10 model is trained in a notebook and exported as `.mem` files with SHA-256 hashes.
Pre-processing / Interface layer	5-stage RV32IM + Zicsr + Zifencei + Zbb core decodes the custom-0 instructions and pushes LOAD_WEIGHT, LOAD_INPUT, and COMPUTE commands from the MEM stage into the 8-entry command FIFO; READ_RESULT stalls until the engine has drained.
Proposed core module	`mac_engine`: FSM (IDLE, CHECK, LOAD, MAC); operand buffers WBUF and IBUF (256 × 128 bit); 16 dot4 PEs and a 4×4 int32 accumulator tile (wrap modulo 2^{32}); CSRs npu_status and npu_busy_cyc. Operands come from DTCM port B (128 bit, read-only).
Optimization / Control / Scheduling	Command FIFO and engine FSM: every command is validated in the CHECK state (illegal operation, K range and multiple of 4, alignment, address range); invalid commands are dropped and recorded in sticky error bits; READ_RESULT acts as a barrier that waits until the FIFO is empty and the engine idle. A software rule prevents overwriting DTCM data that a queued LOAD still reads.
Output / Decision / Actuation	READ_RESULT returns accumulator entries to a scalar register; firmware adds the bias, requantizes with clamp((acc·M + 2^(S−1)) >> S, 0, 127), and takes the argmax of the first 10 of 12 padded logits. The test ends with a write to SIM_CTRL.
Monitoring / Evaluation interface	Three UVM environments (mac_env, core_env, soc_env) with C golden models over DPI-C (npu_ref and an RV32 instruction-set simulator) and a Python integer model of the MLP, SystemVerilog assertions, functional coverage, and a passive retire-trace agent that compares every retired instruction with the instruction-set simulator.

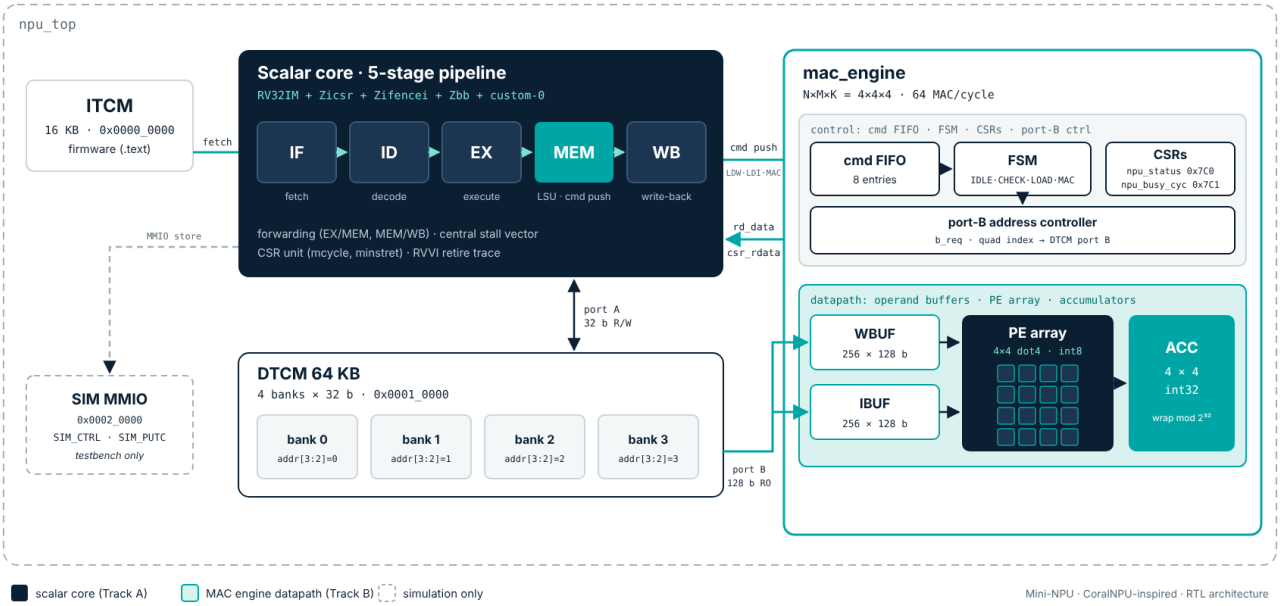


Figure 1. Mini-NPU system architecture (npu_top). The scalar core (dark) pushes commands to the decoupled MAC engine (teal), and both share a 64 KB dual-port DTCM. The SIM MMIO block exists in simulation only.

3.3 Novelty and Expected Technical Contributions

Contribution 1. A working small-scale RISC-V NPU in RTL on the Coral NPU architecture: a custom-0 instruction extension, a command FIFO, a decoupled int8 outer-product engine, and a dual-port DTCM.

Contribution 2. A self-built UVM verification infrastructure (three environments, DPI-C golden models, assertions, and functional coverage) with its closure evidence, checked for sensitivity by injected faults.

Contribution 3. An end-to-end demonstration: a quantized 784-64-32-10 MLP, with its data pipeline and firmware, runs on the NPU and matches the golden model bit-exactly on MNIST.

Contribution 4. A reproducibility package that regenerates the regression, coverage, and MNIST results with one command per flow.

The novelty lies in the combination and in the evidence, not in a new architecture (Section 2.4).

3.4 Hypotheses

Hypothesis	Testable Statement	Evidence Required
H1	The NPU passes the full verification suite (engine, core, and SoC) with zero scoreboard mismatches, zero assertion failures, and 100 % of the declared coverage bins.	Regression logs with UVM_ERROR and UVM_FATAL counts; merged coverage report
H2	The UVM environments are sensitive: each deliberately injected fault (a self-test or a mutated RTL or testbench) makes the environment fail for the stated reason.	Expected-fail self-tests and fault-injection results
H3	The MNIST MLP runs on the NPU with logits and labels bit-exact against the golden model on the first 1,000 test images, and the full run reproduces the golden accuracy.	Golden dump compared with the RTL LOGITS dump; accuracy on the 10,000 test images

3.5 Scope

IN SCOPE:

– The core extensions M, Zicsr, Zifencei, and Zbb, and the four custom instructions with their command FIFO.

– The MAC engine (FIFO, operand buffers, PE array, accumulators, CSRs), the dual-port DTCM, the ITCM, and the simulation MMIO.
– Three UVM environments, a C golden model of the engine, a C instruction-set simulator, and an integer golden model of the MLP.
– Bare-metal firmware and the MNIST pipeline (training, quantization, data packing, firmware, and the bit-exact comparison).

OUT OF SCOPE / NON-GOALS (deferred to the graduation thesis):

– Clock gating and any power or energy estimation or optimization; synthesis, place-and-route, and FPGA or board bring-up.
– RVV/Zve32x and the F extension; DMA, multiple accumulator tiles, and a pipelined PE.
– A compiler toolchain (IREE or TensorFlow Lite Micro [15]) and frontdoor (bus) loading of programs.
– LLM-assisted verification. The 8×8 array is built only if time remains after the core criteria are met.

4 METHODOLOGY / IMPLEMENTATION AND EVALUATION PLAN

Explain how the proposed idea will be built, tested, compared, and analyzed.

4.1 Overall Research Methodology

Phase	Main Activities	Output / Evidence
A. Problem and baseline definition	Fix the scope; freeze the custom ISA and the engine interface; check the tools (UVM, DPI-C, coverage); write the golden model of the engine with unit tests.	Design spec with its decision log; frozen interface; tool-gate results
B. Design	Extend the core (M, Zicsr, Zifencei, Zbb, custom-0, retire trace); write the dual-port DTCM, the MAC engine, and the SoC top; write the firmware base.	RTL; block notes; directed-test and regression results
C. Implementation and verification	Build mac_env, core_env, and soc_env; write constrained-random sequences (200 or more seeds), assertions, and coverage; validate the checks by fault injection.	Regression logs; merged coverage report (SC1, SC2); fault-injection results
D. Model demonstration	Train and quantize the MLP; pack weights and inputs into the DTCM image; write the firmware; run MNIST images on the NPU and compare with the golden model.	Export files with hashes; firmware; bit-exact comparison on 1,000 images and accuracy on 10,000 images (SC3)
E. Analysis	Measure cycles per inference and the engine busy fraction; document limitations; write the report and the reproducibility package.	Figures and tables; English report; slides; one-command reproduction (SC4)

4.2 Implementation Plan

Step	Input	Processing / Design Action	Output
1	CoralNPU documentation; course requirements	Study the open design; write the spec, decision log, and success criteria	Approved spec; frozen ISA and interface
2	Spec sections 6, 7, 9	Write the golden models: npu_ref (C), an RV32 instruction-set simulator (C), and an integer MLP model (Python); gate each with its own unit tests	Oracles that pass their unit tests
3	Existing 5-stage RV32I core	Add M, Zicsr, Zifencei, Zbb, custom-0 decode and push, ITCM and DTCM, and the retire-trace port; check against a Python reference ISS	Core regression green
4	Frozen engine interface	Write the command FIFO, FSM, LOAD and MAC states, PE array, CSRs, and reset; measure latency per command	MAC engine; directed suite green
5	Core and engine RTL	Integrate them in the SoC top; write firmware tests	SoC smoke tests
6	Directed and random plans; coverage plan	Build the UVM environments; random sequences over 200 or more seeds; assertions A-MAC-1..5; close coverage; inject faults to check the checks	SC1 and SC2 evidence
7	Trained MLP; verified NPU	Export the model; pack weights and inputs; write the MNIST firmware; run 100, 1,000, and 10,000 images and compare logits with the golden model	SC3 evidence; cycles per inference
8	All flows	Make each flow runnable by one command; record tool versions, seeds, and the git hash; test on a clean clone	SC4 reproducibility package

4.3 Hardware and Software Platform

Component / Tool	Version / Specification	Purpose in the Project
Processing platform	Windows PC with WSL2 (Ubuntu 24.04)	Host for QuestaSim and for the Linux tools of the regression
Sensors / Interfaces	Not applicable (simulation only)	No board bring-up in this project
Operating system / RTOS	Windows with WSL2 Ubuntu 24.04; CI on Ubuntu 24.04. No RTOS: bare-metal firmware	Same Linux environment locally and in continuous integration
Programming language	SystemVerilog and Verilog-2001 (RTL, testbenches); C99 (golden models, DPI-C); Python 3; RISC-V assembly and C (firmware, `-march=rv32im_zicsr_zifencei_zbb`)	RTL and verification; golden models and firmware; training and data export
Framework / Library	UVM 1.2 (IEEE 1800.2 [10]); numpy; PyTorch (training only); pytest	Verification methodology; integer reference and its unit tests; MLP training
Development / Simulation tool	QuestaSim 10.7c; Verilator 5.020 (lint); Icarus Verilog 12 (core regression); RISC-V GNU toolchain; GitHub Actions	Simulation and coverage; lint; regression of the core; compilation of firmware; automatic regression on every push

4.4 Data, Workloads, Benchmarks, or Test Scenarios

Item	Definition
Dataset / benchmark / workload	(1) MNIST [12] with a 784-64-32-10 MLP quantized to int8, run by bare-metal firmware. (2) Directed and constrained-random tests for the engine, the core, and the SoC.
Size / number of samples / duration	MNIST: the first 1,000 test images for the bit-exact comparison and the full 10,000-image test set for accuracy; a 100-image run for cycle measurements. Random tests: 200 or more seeds per environment.
Pre-processing / normalization / compilation	Pixels to int8 by (p·254 + 255) // 510. Symmetric int8 weights in ±127, int32 biases. Per-layer requantization with an integer multiplier M and shift S chosen as the largest S for which bound·M + 2^(S−1) ≤ $2^{31}-1$. The last layer is padded from 10 to 12 outputs; argmax ignores the padding.
Train / validation / test split	Training on the 60,000-image training set (seed 0); 10,000 training images calibrate the activation scales (99.99th percentile); evaluation on the 10,000-image test set. Hardware comparisons use test images only.
Operating conditions / input ranges	RTL simulation of one clock domain, no timing back-annotation; the same binary and simulator version for every comparison; random seeds are explicit.
Ground truth / reference output	Independent golden models (engine, instruction-set simulator, integer MLP) and the retire trace of the core; the models are used only after their own unit tests pass.

4.5 Baselines and Comparative Methods

Baseline	Reason for Selection	Configuration / Fairness Controls
Baseline 1: independent golden models	Oracle for functional correctness; bit-exact comparison removes tolerance arguments	Used only after their own unit tests pass; cross-checked against one another (the integer MLP model against the engine model, the instruction-set simulator against a Python reference ISS)
Baseline 2: float MLP	Accuracy reference for the int8 model [1]	Same trained weights, same 10,000 test images

4.6 Evaluation Metrics

Dimension	Candidate Metrics	Selected Metric(s) and Measurement Method
Quality / ML	Accuracy, label agreement	int8 and float test accuracy; logits and labels bit-exact against the golden model

Dimension	Candidate Metrics	Selected Metric(s) and Measurement Method
Performance	Cycles, latency, throughput	mcycle and minstret per inference; busy cycles per engine command (bound K/4 + 3); engine busy fraction = npu_busy_cyc / mcycle. Reported, with no preset target.
Compute / Memory	MACs per cycle, memory footprint	Peak 64 MACs per cycle; DTCM data region 56,816 B of 65,536 B
Hardware	LUT, FF, area, frequency	Not evaluated (no synthesis)
Energy	Power, energy per inference	Not evaluated (deferred to the thesis)
Reliability	Mismatches, assertion failures, coverage	Scoreboard mismatches and assertion failures (target 0); merged functional coverage (target 100 % of declared bins); injected faults detected

4.7 Experimental Design

Experiment	Objective	Independent Variables	Measured Outputs	Decision / Comparison
E1: Functional verification	Show that the engine, the core, and the SoC are correct against the golden models	Environment (mac_env, core_env, soc_env); test (directed, random seed)	Scoreboard mismatches; assertion failures; coverage %	Accepted only with 0 mismatches, 0 failures, and 100 % of declared bins (SC1, SC2)
E2: Checker sensitivity	Show that the checks detect real faults	Injected fault (self-test, mutated RTL, or mutated testbench)	Fail or pass of the run; the reported error ID	Each fault must be caught for the stated reason (H2)
E3: MNIST end to end	Show that the NPU runs the quantized model bit-exactly	Image count (100, 1,000, 10,000)	Logits and labels versus golden; accuracy; mcycle and npu_busy_cyc per inference	Bit-exact (SC3); accuracy equal to the golden accuracy; cycles reported

4.8 Analysis and Reproducibility Plan

- Every simulation runs through one Python runner (options for the simulator, the test target, the seeds, and coverage); logs are kept per run under a dated directory with a `manifest.yaml` that records QuestaSim version and licence type, Verilator, GCC, and Python versions, the git hash, seeds, and the SHA-256 of each log.
- Random seeds are explicit (for example 1..200). Golden models are gated: each passes its own unit tests before it is used as an oracle, and testbenches are checked by injecting faults and confirming that the checks fail.
- Model data is exported with per-file SHA-256 hashes; the integer golden model, the C engine model, and the RTL are compared on the same exported files.
- A mismatch is localized layer by layer (first differing logit, then the activations of the previous layer) using the golden models, so that a failing run reports where the first difference occurs.
- One command per flow regenerates the regression, the coverage, and the MNIST results; this is checked on a clean clone in the final weeks (SC4).

4.9 Technical Risks and Mitigation

Risk	Impact	Mitigation / Contingency
Simulator licence lacks a feature (UVM, randomize, covergroup, DPI-C)	UVM flow blocked	Checked in week 1: QuestaSim 10.7c ran all four. Fallbacks: Vivado xsim (verified then), Verilator, or pyuvm.
Simulation too slow for the MNIST runs	SC3 cannot finish	Measure early. Cut order: the 10,000-image run first, then fewer images for the cycle measurement; the bit-exact run on 1,000 images and the report are never cut.

Risk	Impact	Mitigation / Contingency
Interface changes between the two tracks	Rework in both	Interface frozen at week 2 (both students signed on 2026-10-29); later changes need both signatures and a spec update.
Golden model and RTL share a misreading of the spec	Wrong results accepted	Independent models in C and Python cross-checked against one another; testbench fault injection; hand-computed unit cases.
Firmware overwrites DTCM data that a queued LOAD still reads	Corrupt inputs	Software rule (drain with READ_RESULT before overwriting) plus an assertion and a negative test that proves it fires.
Quantized model differs from the firmware's requantization	MNIST not bit-exact	Requantization formula fixed in the spec; the firmware routine is unit-tested on the host against the golden model before it runs on the NPU.
Verification effort exceeds the schedule	Coverage not closed in time	Directed tests first, random and coverage after; coverage exclusions need a written reason; weeks 13–14 hold buffer; the 8×8 array is dropped first.

5 EXPECTED RESULTS

Define measurable outcomes, expected contributions, and concrete project deliverables.

5.1 Expected Technical Results

The NPU is expected to pass its full verification suite with zero scoreboard mismatches and zero assertion failures and to close 100 % of the declared functional-coverage bins for the engine, the core, and the SoC. The verification checks are expected to detect every deliberately injected fault. The 784-64-32-10 int8 MLP is expected to produce logits and labels that match the golden model bit-exactly on the first 1,000 test images, and to reproduce the golden accuracy (97.43 % with the current model) on the full 10,000-image run. Cycles per inference and the engine busy fraction are measured and reported with no preset target; they show how the run divides between the engine and the scalar firmware.

5.2 Target Performance Table

Metric	Baseline / Current Value	Expected Target	Measurement Condition
Primary metric: functional correctness	mac_env: 224 of 224 runs as expected, 81,332 scoreboard comparisons, 0 mismatches (project log, 2026-11-10)	0 mismatches and 0 assertion failures for the engine, the core, and the SoC	QuestaSim 10.7c, UVM 1.2, runner; E1
Secondary metric: functional coverage	mac_env: 100 % of 7 covergroups merged (project log, 2026-11-10); core_env and soc_env not yet measured	100 % of declared bins for all three environments	Merged coverage report; E1
Resource metric: MNIST accuracy	int8 test accuracy 97.43 % (float 97.48 %) on 10,000 images	RTL logits and labels bit-exact on the first 1,000 images; the full run reproduces the golden accuracy	Golden dump compared with the RTL LOGITS dump; E3
Robustness metric: checker sensitivity	Self-tests for the memory-mirror and write-after-read checks exist in mac_env	Every injected fault caught for the stated reason	Fault-injection runs; E2
Reproducibility metric	Per-run manifest exists; one command for the engine regression	One command per flow, verified on a clean clone	Final weeks; SC4

5.3 Expected Scientific / Engineering Contributions

- A small, fully explained RISC-V NPU on the Coral NPU architecture (custom-instruction extension, decoupled int8 engine, dual-port DTCM) with its design decisions recorded.
- A self-built UVM verification environment with independent golden models, assertions, closed coverage, and evidence that its checks detect faults.
- An end-to-end demonstration that a quantized MLP runs on the NPU bit-exactly, including the data pipeline and firmware.
- A reproducibility package and documented design that can serve as the base for later work such as the graduation thesis.

5.4 Expected Deliverables

Deliverable	Minimum Content / Acceptance Evidence
Research proposal	Approved problem, gap, idea, methodology, and expected results
Literature review	Critical synthesis of the related work (Section 2) and the related-work part of the final report
Design artifacts	Design spec with its decision log, block notes for the core and the MAC engine, the architecture figure, and the interface definitions
Prototype	RTL of the core, MAC engine, dual-port DTCM, and SoC top; firmware; golden models

Deliverable	Minimum Content / Acceptance Evidence
Experiment package	UVM environments, test lists, the runner, logs with manifests, coverage reports, and the MNIST comparison files
Technical report	English report with verification and MNIST chapters, discussion of limitations, and conclusion
Final presentation and demo	Live regression, MNIST inference on the NPU with the bit-exact comparison, and the coverage results
Repository	GitHub repository with RTL, testbenches, models, firmware, scripts, documents, and the reproducibility package

5.5 Interpretation of Negative or Mixed Results

- If some coverage bins cannot be reached, each is analysed and either closed with a new test or excluded with a written reason; the analysis shows which behaviour the specification makes unreachable and is itself a verification result.
- If the MNIST run is not bit-exact at first, the layer-by-layer comparison localizes the first difference to the engine, the requantization, or the data packing; a documented root cause is still a valid result and a regression test is added for it.
- If cycles per inference are high, the split between engine busy cycles and scalar firmware cycles shows where the time goes, which is useful evidence for future work on the NPU (for example moving requantization into hardware).
- All results come from RTL simulation of one design and one simulator. They are evidence about this design, not claims about silicon or about NPUs in general.

Project Timeline and Milestones

Dates follow the project calendar: week 1 starts Monday 12 October 2026 and week 16 ends Sunday 31 January 2027. Status is given as of 11 November 2026 (week 5).

Week	Activities	Milestone / Deliverable (status 2026-11-11)
1 12–18 Oct	Tool gate: QuestaSim UVM, DPI-C, coverage, toolchain; problem scoping	Tool-gate results. *Done 2026-10-18.*
2 19–25 Oct	ISA and CSR documents; engine interface; engine golden model with unit tests	Frozen ISA and interface. *Golden model done 2026-10-17; interface signed by both on 2026-10-29.*
3–4 26 Oct–8 Nov	M, Zicsr, Zifencei, Zbb, custom-0 decode and push; instruction-set simulator; MAC engine; dual-port DTCM	Phase 1 exit. *Core regression 1,291 of 1,291; engine directed suite green on QuestaSim (2026-11-05); one core policy decision (W2-1) still open.*
5–6 9–22 Nov	SoC integration and firmware base; mac_env and core_env directed tests	First soc test. *Engine-side integration with 8 assembly programs passing (2026-11-10); mac_env directed and random done (2026-11-07).*
7–9 23 Nov–13 Dec	Random tests and coverage; soc tests; fault-injection checks; coverage closure	SC1 and SC2; verification chapter draft. *mac_env: 100 % of 7 covergroups (2026-11-10); core_env and soc_env pending.*
10 14–20 Dec	MNIST pipeline: weight and input packing, golden files, firmware	Packed DTCM image; first MNIST images on the NPU
11 21–27 Dec	MNIST on the first 1,000 test images; layer-by-layer debugging	SC3 (bit-exact on 1,000 images)

Week	Activities	Milestone / Deliverable (status 2026-11-11)
12 28 Dec–3 Jan	Full 10,000-image run; cycles per inference and engine busy fraction	Golden accuracy reproduced; performance table
13–14 4–17 Jan	Buffer for coverage or MNIST fixes; verification and MNIST report chapters; reproduction on a clean clone	SC4; chapter drafts
15 18–24 Jan	Report and slides; final regression	Draft report and slides
16 25–31 Jan	Final validation, demo, and submission	Final report, demo, and repository

References

[1] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a Tensor Processing Unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017, pp. 1–12.

[3] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[5] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, 2014, pp. 10–14.

[6] Google, "Coral NPU," GitHub repository *google-coral/coralnpu*, commit 382b5c1, and Coral NPU developer guides (developers.google.com/coral/guides), read Oct. 15, 2026.

[7] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, 2021, pp. 769–774.

[8] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, Dec. 2019.

[9] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, pp. 48–60, Feb. 2019.

[10] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2017, 2017.

[11] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[13] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2021.

[14] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. Pearson, 2019.

[15] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021, pp. 800–811.